2

Lab

Lập trình ngôn ngữ Assembly cơ bản

Basic Assembly Programming

Thực hành Lập trình Hệ thống

Lưu hành nội bộ

A. TỔNG QUAN

A.1 Mục tiêu

- Tìm hiểu và làm quen với Hợp ngữ (Assembly Language - ASM)
- Tìm hiểu quá trình dịch từ một mã nguồn assembly thành tập tin thực thi
- Thực hành viết một số chương trình mẫu dưới dạng hợp ngữ (theo AT&T)

A.2 Môi trường

- Máy cài Hệ điều hành Linux (máy ảo).
- Các công cụ biên dịch, hợp dịch, liên kết gồm **gcc**, **as**, **ld**.

A.3 Liên quan

- Sinh viên cần vận dụng kiến thức trong Chương 3 (Lý thuyết).
- Các kiến thức này đã được giới thiệu trong nội dung lý thuyết đã học do đó sẽ không được trình bày lại trong nội dung thực hành này.
- Tham khảo tài liệu (Mục F).

B. KIẾN THỨC NỀN TẢNG

B.1 Hợp ngữ là gì?

Hợp ngữ - Assembly Language (hay viết tắt là ASM) là ngôn ngữ bậc thấp, và nằm ở vị trí trung gian bên cạnh ngôn ngữ máy và ngôn ngữ bậc cao.

ASM sử dụng các từ gợi nhớ (mnemonics) để viết các chỉ thị (instructions) lập trình cho máy tính thay vì bằng những dãy số 0 và 1 (ngôn ngữ máy).

```
push %ebp
mov  %esp, %ebp
sub  $0x8, %esp
movl $0x1, -4(%ebp)
sub  $0xc, %esp
push $0x0
call 8048348
```

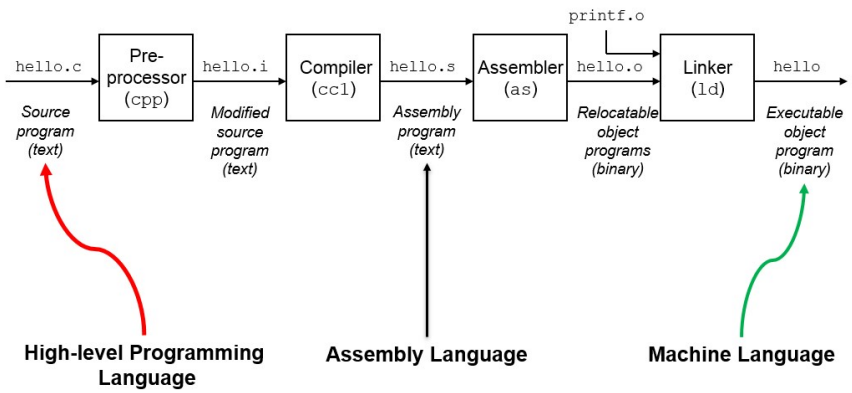
Hình 1. Ví dụ về một đoạn hợp ngữ.

Về **ưu điểm**, Hợp ngữ thân thiện hơn so với ngôn ngữ máy – vốn rất khó hiểu, dễ gây lỗi và tốn nhiều thời giờ. Hợp ngữ có thể tương tác rất sâu dưới hệ thống, chúng có thể giao tiếp trực tiếp với các phần cứng và bắt chúng hoạt động theo ý người lập trình. Bên cạnh đó, hợp ngữ cũng rất hữu ích trong kỹ thuật dịch ngược (reverse engineering) – được sử dụng rộng rãi trong lĩnh vực an toàn thông tin.

Về **nhược điểm**, so với các ngôn ngữ bậc cao quen thuộc (C, C++, Java, Python …), hợp ngữ ít thân thiện hơn, và phụ thuộc vào kiến trúc CPU (ARM, x86-32, x86-64), hệ điều hành (Linux, Windows, Mac) và các tập chỉ thị mà nhà sản xuất phần cứng đưa ra. Nói cách khác, ASM là một ngôn ngữ không giống như các ngôn ngữ lập trình khác, không có một định dạng chuẩn nào cho các trình hợp dịch (Assembler) sử dụng để dịch các chương trình ASM.

Một chương trình viết bằng ngôn ngữ bậc cao hay hợp ngữ không thể được thực thi trực tiếp bởi máy tính. Sau khi được viết xong, chương trình này phải trải qua quá trình dịch thành ngôn ngữ máy. Quá trình này được mô tả trong **Hình 2** bao gồm các giai đoạn:

- Giai đoạn tiền xử lý (Pre-processor)
- Giai đoạn dịch từ ngôn ngữ bậc cao sang hợp ngữ (Compiler)
- Giai đoạn dịch hợp ngữ sang ngôn ngữ máy (Assembler)
- Giai đoạn liên kết (Linker)



Hình 2. Quá trình dịch từ 1 chương trình dạng C sang tập tin thực thi.

Trong bài thực hành này, sinh viên được yêu cầu viết hợp ngữ theo định dạng AT&T (sử dụng Hệ điều hành Linux, các công cụ biên dịch, hợp dịch như gcc, as, ld)

B.2 Cấu trúc cơ bản và các thành phần của một chương trình hợp ngữ

B.2.1 Cấu trúc của chương trình hợp ngữ

Trong Linux, các chương trình viết bằng mã assembly sẽ có đuôi **.s**. Thông thường chương trình hợp ngữ có ba phần (section) được khai báo:

B.2.1.1 Section .data

Là section khai báo những **vùng nhớ** hoặc **hằng số**, nơi chứa các dữ liệu dùng cho chương trình. Thường sử dụng để khai báo biến đã được khởi tạo giá trị hoặc các hằng số sử dụng cho chương trình. Định dạng khai báo:

```
.section .data
<name1>:  .<type> <data1>     # vùng nhớ chứa dữ liệu
<name2> = <data2>             # hằng số
...
```

Trong đó:

+ **name** là tên gợi nhớ, có thể được dùng ở các lệnh để truy xuất vùng nhớ hoặc hằng số sau này.
+ **type** là kiểu dữ liệu, ví dụ: int, string, byte...
+ **data** là giá trị cụ thể cần khởi tạo cho vùng nhớ trong vùng nhớ.

Ví dụ: đoạn mã khai báo vùng nhớ **a** lưu số nguyên **10**, vùng nhớ **my_str** lưu chuỗi **"Hello"** và 1 hằng số **b** có giá trị 9.

```
.section .data
a:      .int 10
my_str: .string "Hello"
b = 9
```

B.2.1.2 Section .bss

Phần .bss khai báo các dữ liệu chưa được gán giá trị (hoặc null), thường được dùng khi cần chuẩn bị sẵn các **vùng nhớ** trống để lưu dữ liệu sau này. Đây là thành phần không bắt buộc, nếu không sử dụng có thể bỏ qua. Định dạng khai báo:

```
.section .bss
      .lcomm <name1>, <length>
      .lcomm <name2>, <length>
      ...
```

Trong đó:

+ **name** là tên gợi nhớ, có thể được dùng ở các lệnh để truy xuất vùng nhớ này.
+ **length** là kích thước vùng nhớ cần được cấp, tính theo byte.

Ví dụ: đoạn mã bên dưới khai báo vùng nhớ tên **mem** gồm 2 bytes và vùng nhớ **result** có kích thước 4 bytes.

```
.section .bss
      .lcomm mem, 2
      .lcomm result, 4
```

B.2.1.3 Section .text

Là section **bắt buộc** phải có trong tất cả các chương trình hợp ngữ. Đây là nơi các câu lệnh được khai báo. Định dạng khai báo:

```
.section .text
      .globl _start

_start:
      // các lệnh assembly
```

Sau nhãn _start, các câu lệnh assembly sẽ được viết tùy thuộc vào chức năng cần lập trình. Trong đó có thể sử dụng các vùng nhớ đã được khai báo ở 2 section .data hoặc .bss để lấy hoặc lưu trữ dữ liệu vào các vùng nhớ nhất định, hoặc tương tác với các thanh ghi.

Với 1 vùng nhớ tên **a** được khai báo trong section .data (hoặc .bss):

- Ký hiệu **$a** tương ứng với việc lấy *địa chỉ* của vùng nhớ **a,** được xem là 1 hằng số.
- Ký hiệu **(a)** hoặc **a** tương ứng với việc truy xuất *giá trị* đang lưu tại vùng nhớ **a**.

B.2.2 Định dạng lệnh assembly

Trong phạm vi bài thực hành này, các lệnh assembly được viết theo định dạng AT&T và tập lệnh của kiến trúc 32bit.

B.3 Linux System Call (Hàm gọi hệ thống Linux)

Có nhiều lời gọi hệ thống (system call) được cung cấp bởi kernel Linux và biết cách tìm cũng như sử dụng chúng rất có lợi trong việc lập trình hợp ngữ (assembly language). Những system call này có sẵn cho các lập trình viên sử dụng. Thông thường, với mỗi lần phát hành một kernel mới, các system call mới được thêm vào danh sách.

Các system call thường được định nghĩa trong file sau:
/usr/include/asm/unistd.h hoặc */usr/include/asm-generic/unistd.h*

Không giống như các hàm kiểu C, trong đó các giá trị đầu vào được đặt trên stack, các system call yêu cầu đầu vào sao cho các giá trị được đặt trong thanh ghi. Có một thứ tự cụ thể trong đó mỗi giá trị đầu vào được đặt trong các thanh ghi tương ứng. Đặt giá trị đầu vào trong một thanh ghi không đúng chuẩn có thể tạo ra kết quả sai.

Thứ tự trong đó các system call mong đợi các giá trị đầu vào như sau:

- %eax
- %ebx (tham số đầu tiên)
- %ecx (tham số thứ hai)
- %edx (tham số thứ ba)
- %esi (tham số thứ tư)
- %edi (tham số thứ năm)

Sử dụng quy ước này, các giá trị đầu vào sẽ được gán cho các thanh ghi sau:

- %eax: Giá trị của hàm gọi hệ thống (tham khảo bảng bên dưới)
- %ebx: Bộ mô tả tệp (số nguyên)
- %ecx: Con trỏ (địa chỉ bộ nhớ) của chuỗi cần hiển thị
- %edx: Kích thước của chuỗi cần hiển thị

%eax	Name	%ebx	%ecx	%edx	%esx	%edi
1	sys_exit	int	-	-	-	-
2	sys_fork	struct pt_regs	-	-	-	-
3	sys_read	unsigned int	char *	size_t	-	-
4	sys_write	unsigned int	const char *	size_t	-	-
5	sys_open	const char *	int	int	-	-
6	sys_close	unsigned int	-	-	-	-

Giá trị mô tả tệp cho vị trí đầu ra (hoặc vào) để chỉ định vị trí nhận hoặc xuất dữ liệu được đặt trong %ebx. Các hệ thống Linux chứa ba mô tả tệp đặc biệt:

- 0 (STDIN): Đầu vào tiêu chuẩn cho thiết bị đầu cuối (thường là bàn phím)
- 1 (STDOUT): Đầu ra tiêu chuẩn cho thiết bị đầu cuối (thường là màn hình đầu cuối)
- 2 (STDERR): Đầu ra lỗi tiêu chuẩn cho thiết bị đầu cuối (thường là màn hình đầu cuối)

Ví dụ để gọi hàm in ra màn hình chuỗi "Hello, World".

- **%eax = 4** tương ứng với chỉ thị in chuỗi.
- **%ebx = 1** tương ứng với vị trí in ra là stdout (terminal).
- **%ecx = địa chỉ ô nhớ** đang chứa chuỗi cần in.
- **%edx = độ dài** (tính theo byte) của chuỗi cần in.

Sau khi khai báo các giá trị tham số cần thiết của system call trong các thanh ghi, lệnh **int $0x80** được dùng để thực thi system call đó.

```
.section .data
output:
    .string "Hello, World"

.section .text
    .globl _start
_start:
    movl $13, %edx          # message length
    movl $output, %ecx      # address of message to write.
    movl $1, %ebx           # file descriptor (stdout)
    movl $4, %eax           # system call number (sys_write)
    int $0x80               # call kernel

    movl $1, %eax           # system call number (sys_exit)
    int $0x80               # call kernel
```

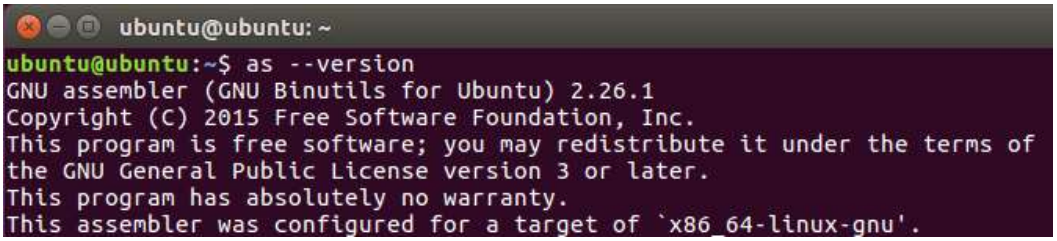
C. THỰC HÀNH

C.1 Yêu cầu 1 – Thiết lập môi trường

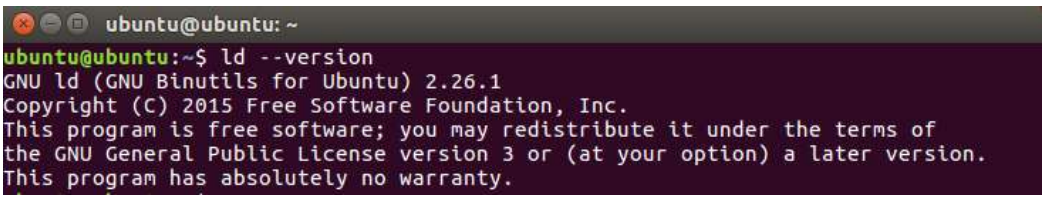
Sinh viên cần chuẩn bị môi trường máy ảo Linux có đầy đủ các công cụ cần thiết bao gồm **as** và **ld** để sử dụng trong quá trình biên dịch file mã hợp ngữ sang file thực thi.

Các công cụ trên có thể được kiểm tra đã được cài đặt trên hệ thống chưa bằng việc gõ các lệnh sau trong terminal:

```
as --version
```




```
ld --version
```



Nếu có kết quả nào trả về không tìm thấy công cụ, sinh viên gõ lệnh để cài đặt:

```
sudo apt-get install binutils
```

C.2 Yêu cầu 2 – Bài tập lập trình hợp ngữ (assembly)

Hướng dẫn chung: Sinh viên thực hiện viết các chương trình dưới dạng hợp ngữ (assembly) trong các file **.s**, sau đó thực hiện các bước hợp dịch và liên kết như sau:

- Hợp dịch với công cụ **as** để thu được một file nhị phân **.o** từ file **.s**:

```
as -o <file .o> <file .s đầu vào>
```

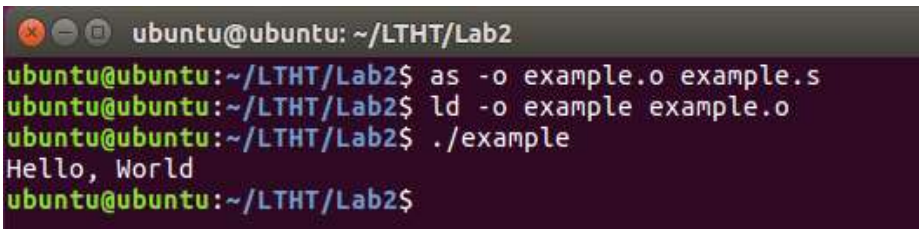
- Liên kết với công cụ **ld** để tạo file thực thi của chương trình

```
ld -o <file thực thi> <file .o đầu vào>
```

- Thực thi file

```
./<file thực thi đầu ra>
```

Ví dụ:



C.2.1 Chương trình in ra độ dài của một chuỗi cho trước (tối đa 9 ký tự)

Input: Chuỗi **msg** được khai báo sẵn trong section **.data,** tối đa 9 ký tự.

Output: Xuất ra màn hình độ dài của chuỗi (số ký tự - không tính ký tự null).

Ví dụ cơ bản: Với chuỗi **msg = "Love UIT"**, kết quả đầu ra như hình dưới:

```
ubuntu@ubuntu: ~/LTHT/Lab2/ANTN/C21
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C21$ as C21.s -o C21.o
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C21$ ld C21.o -o C21
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C21$ ./C21
8
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C21$
```

Gợi ý 1.1: Phần **D.3** hướng dẫn cách lấy được độ dài của một chuỗi đã khai báo trong section **.data**. *Lưu ý độ dài này sẽ có tính luôn ký tự null ở cuối chuỗi.*

Gợi ý 1.2: Để in một dữ liệu nào đó ra màn hình, cần đảm bảo 2 yếu tố:

- Dữ liệu cần in cần **nằm trong 1 vùng nhớ** (trong section .bss hoặc .data), không hỗ trợ in trực tiếp giá trị từ thanh ghi hay hằng số.
- Khi in, hệ thống tự động xem dữ liệu được cung cấp là **các mã ASCII** của 1 hoặc nhiều ký tự nào đó và in ra các ký tự tương ứng. Do vậy, nếu dữ liệu không phải là 1 mã ASCII in được thì khi in sẽ không nhìn thấy được. Ví dụ: Khi dữ liệu là 65 thì hệ thống sẽ in ra ký tự 'A', còn dữ liệu là 0 thì không in ra được gì.

Gợi ý 1.3: Ta cần thiết lập giá trị các thanh ghi cho việc xuất dữ liệu như sau:

- %eax: SYS_WRITE (4).
- %ebx: với STDOUT (1).
- %ecx: ***địa chỉ ô nhớ*** có dữ liệu cần xuất
- %edx: độ dài sẽ xuất ra (bytes).

C.2.2 Chương trình tính số dư của tổng 4 số có 2 chữ số khi chia 8

Input: Nhập từ bàn phím 4 số nguyên có 2 chữ số ($10 \leq x \leq 99$).

Output: Số dư khi chia tổng 4 số đã nhập cho 8.

Ví dụ:

```
ubuntu@ubuntu: ~/LTHT/Lab2/ANTN/C22
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C22$ as C22.s -o C22.o
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C22$ ld C22.o -o C22
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C22$ ./C22
Enter a number (2-digit): 34
Enter a number (2-digit): 18
Enter a number (2-digit): 20
Enter a number (2-digit): 10
Reminder: 2
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C22$ ./C22
Enter a number (2-digit): 15
Enter a number (2-digit): 15
Enter a number (2-digit): 15
Enter a number (2-digit): 16
Reminder: 5
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C22$
```

Gợi ý 2.1: Để nhận input từ bàn phím, ta thực hiện các bước:

- Chuẩn bị sẵn 1 vùng nhớ trong section .bss có độ dài phù hợp để lưu input.

*Ví dụ bên dưới tạo 1 vùng nhớ có kích thước **2 bytes** (lưu 2 ký tự) có nhãn là **number1**.*

```
.section .bss
        .lcomm number1, 2
```

Trong đó: **$number1** sẽ lấy địa chỉ vùng nhớ, **(number1)** sẽ lấy giá trị trong vùng nhớ.

- Ta cần thiết lập giá trị các thanh ghi cho 1 system call nhập dữ liệu như sau:
  - %eax = 3 (SYS_READ).
  - %ebx = 0 (STDIN).
  - %ecx: ***địa chỉ ô nhớ*** để lưu giá trị nhập vào (đã tạo trong section .bss)
  - %edx: độ dài của chuỗi sẽ nhận (tính theo bytes)

Lưu ý: Độ dài nhập vào cần tính cả ký tự xuống dòng. Do đó, với input là 1 ký tự hay số 1 chữ số cần khai báo độ dài của chuỗi nhập vào trong %edx là 2.

Gợi ý 2.2: 1 ký tự chỉ ứng với 1 byte, chỉ nên lấy đủ 1 byte tương ứng với ký tự đã nhập khi cần xử lý 1 ký tự. Dựa vào địa chỉ của chuỗi để truy xuất các ký tự dựa trên chỉ số (index) của chúng. Ví dụ bên dưới đoạn mã lấy ký tự thứ 2 (chỉ số index = 1) của chuỗi **input**.

```
    movl $input, %eax
    movb 1(%eax), %bl
```

Gợi ý 2.3: Dữ liệu nhập vào là các ký tự (ký tự chữ hoặc số). Do vậy sau khi nhập số từ bàn phím, cần chuyển từ ký tự số sang giá trị số nguyên để tính toán (ví dụ: chuyển từ '1' sang 1, chuyển từ chuỗi số '12' sang số nguyên 12, xem phần **D.2**).

Gợi ý 2.4: Có thể tính số dư của phép chia cho 8 bằng phép tính toán trên bit hoặc dùng lệnh lệnh div (x86 Intel).

C.2.3 Chương trình thực hiện mã hóa Caesar trên 1 chuỗi

Input: Chuỗi **5 ký tự** in thường, và số bước dịch **n** (0 – 9).

Output: Xuất chuỗi ký tự đã được mã hóa Caesar với bước dịch n.

Giới hạn cơ bản: Các ký tự thường x và số bước dịch n sao cho 'a' ≤ x + n ≤ 'z'.

Nâng cao 1: Có xử lý xoay vòng khi ký tự được dịch vượt quá 'z'.

Nâng cao 2: Mã hóa tất cả các ký tự hoa và thường, ký tự số và ký tự đặc biệt không đổi, chuỗi có độ dài tùy chọn và tối đa 255 ký tự.

Ví dụ (cơ bản):

```
ubuntu@ubuntu: ~/LTHT/Lab2/ANTN/C23
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C23$ as C23.s -o C23.o
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C23$ ld C23.o -o C23
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C23$ ./C23
Enter a string (<255 chars): abcde
Enter n (0-9): 1
bcdef
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C23$ ./C23
Enter a string (<255 chars): uname
Enter n (0-9): 3
xqdph
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C23$
```

Kết quả nâng cao (1 + 2):

```
ubuntu@ubuntu: ~/LTHT/Lab2/ANTN/C23
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C23$ as C23.s -o C23.o
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C23$ ld C23.o -o C23
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C23$ ./C23
Enter a string (<255 chars): name uit12y TEST@
Enter n (0-9): 3
qdph xlw12b WHVW@
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C23$ ./C23
Enter a string (<255 chars): hello ANTN
Enter n (0-9): 1
ifmmp BOUO
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C23$
```

Gợi ý 3.1: Dùng system call READ để đọc dữ liệu (như C2.2).

Gợi ý 3.2: Sau khi nhập n, cần chuyển n từ ký tự số sang giá trị số nguyên để tính toán (ví dụ: chuyển từ '1' sang 1, xem phần **D.2**).

Gợi ý 3.3 (nâng cao 1): Xem thêm mã hóa Caesar: Caesar cipher.

Gợi ý 3.4 (nâng cao 2): Khi nhập chuỗi, ta thường gõ xuống dòng để kết thúc chuỗi và hệ thống tương ứng sẽ nhận và lưu 1 ký tự ***newline***. Do vậy, ký tự này có thể được dùng làm dấu hiệu để nhận biết vị trí kết thúc của chuỗi đã nhập.

C.2.4 Chương trình đếm số lượng bội của 10 trong khoảng [x,y]

Input: 2 số nguyên 2 chữ số x, y nhập từ bàn phím ($10 \leq x \leq y \leq 99$)

Output: Xuất ra màn hình số lượng bội của 10 trong khoảng [x,y] (có tính x và y).

Ví dụ:

```
ubuntu@ubuntu: ~/LTHT/Lab2/ANTN/C24
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ as C24.s -o C24.o
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ ld C24.o -o C24
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ ./C24
Enter x: 14
Enter y: 35
Count 10x: 2
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ ./C24
Enter x: 10
Enter y: 99
Count 10x: 9
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$
```

Gợi ý 4.1: Khi viết nhiều system call read liên tục để đọc nhiều input, cần khai báo độ dài của dữ liệu nhận vào (giá trị %edx) lớn hơn 1 để tránh ký tự xuống dòng của lần nhập trước ảnh hưởng đến lần nhập sau. Ví dụ: để nhập số có 2 chữ số, cần khai báo %edx = 3.

Gợi ý 4.2: Dữ liệu nhập vào là chuỗi ký tự số, nên chuyển từ ký tự số sang số nguyên để kiểm tra chính xác (ví dụ: chuyển từ '1' sang 1, chuyển '20' sang số 20, *xem phần **D.2***).

C.2.5 Kiểm tra tuổi đoàn dựa vào năm sinh

Input: Năm sinh có 4 chữ số, được nhập từ bàn phím

Output: Xuất ra màn hình: **"Thuoc Doan TN"** nếu tuổi từ **16 đến 30**. Ngược lại xuất ra "**Khong thuoc Doan TN**"

Nâng cao: Xuất kèm số tuổi tính được

Ví dụ (cơ bản):

```
ubuntu@ubuntu: ~/LTHT/Lab2/ANTN/C24
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ as C24.s -o C24.o
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ ld C24.o -o C24
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ ./C24
Nhap nam sinh (4 ky tu): 1995
Khong thuoc Doan TN
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ ./C24
Nhap nam sinh (4 ky tu): 2010
Thuoc Doan TN
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ ./C24
Nhap nam sinh (4 ky tu): 2011
Khong thuoc Doan TN
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$
```

Phần nâng cao:

```
ubuntu@ubuntu: ~/LTHT/Lab2/ANTN/C24
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ as C24.s -o C24.o
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ ld C24.o -o C24
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ ./C24
Nhap nam sinh (4 ky tu): 1995
Khong thuoc Doan TN
Tuoi: 31
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ ./C24
Nhap nam sinh (4 ky tu): 2010
Thuoc Doan TN
Tuoi: 16
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$ ./C24
Nhap nam sinh (4 ky tu): 2012
Khong thuoc Doan TN
Tuoi: 14
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C24$
```

C.2.6 Kiểm tra năm nhuận

Input: Nhập một năm a có 4 chữ số (1000 ≤ a ≤ 9999)

Output: Xuất ra màn hình nhận định: "Nam nhuan" nếu **a là năm nhuận**, ngược lại thì xuất "Khong phai nam nhuan".

Nâng cao: In ra thêm năm nhuận kế tiếp.

Ví dụ:

```
ubuntu@ubuntu: ~/LTHT/Lab2/ANTN/C25
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$ as C5.s -o C5.o
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$ ld C5.o -o C5
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$ ./C5
Enter a year (4-digits): 1999
Khong phai nam nhuan
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$ ./C5
Enter a year (4-digits): 2000
Nam nhuan
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$ ./C5
Enter a year (4-digits): 2100
Khong phai nam nhuan
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$ ./C5
Enter a year (4-digits): 2005
Khong phai nam nhuan
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$
```

Phần nâng cao: In ra thêm năm nhuận kế tiếp

```
ubuntu@ubuntu: ~/LTHT/Lab2/ANTN/C25
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$ as C5.s -o C5.o
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$ ld C5.o -o C5
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$ ./C5
Enter a year (4-digits): 2000
Nam nhuan
Nam nhuan ke tiep: 2004
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$ ./C5
Enter a year (4-digits): 1996
Nam nhuan
Nam nhuan ke tiep: 2000
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$ ./C5
Enter a year (4-digits): 2099
Khong phai nam nhuan
Nam nhuan ke tiep: 2104
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$ ./C5
Enter a year (4-digits): 9999
Khong phai nam nhuan
Nam nhuan ke tiep: 10000
ubuntu@ubuntu:~/LTHT/Lab2/ANTN/C25$
```

D. THAM KHẢO

D.1 Một số lưu ý khi lập trình assembly định dạng AT&T

- **Cần phân biệt được các toán hạng:**

Khác biệt trong ký hiệu của các dạng toán dạng khác nhau:

- Hằng số: **$1**
- Thanh ghi: **%eax**
- Địa chỉ ô nhớ: **$output** với output là nhãn của ô nhớ trong .data hoặc .bss
- Giá trị trong ô nhớ: **(output)** hoặc **output**

- **Trong các câu lệnh assembly, cần đảm bảo:**

  - Có **tối đa 1 toán hạng là hằng số**, nếu có hằng số cần đứng ở vị trí src.
    Ví dụ: addl $1, %eax
  - Có **tối đa 1 toán hạng liên quan đến truy xuất ô nhớ**.

- **Cách viết biểu thức địa chỉ để truy xuất ô nhớ**

  Giả sử với ô nhớ có nhãn tên **output**

  - Truy xuất giá trị ô nhớ từ địa chỉ bắt đầu: viết dưới dạng **(output)** hoặc **output**.
  - Truy xuất giá trị ô nhớ từ địa chỉ cách n bytes: cần đưa địa chỉ vào thanh ghi và dùng biểu thức tính toán địa chỉ

```
    movl $output, %eax
    movl 1(%eax), %ebx    // truy xuất từ địa chỉ cách 1 byte
```

D.2 Bảng mã ASCII

- **Chuyển đổi số có 1 chữ số**

Sử dụng bảng mã ASCII để chuyển đổi từ số (decimal) sang chữ (dạng ascii) và ngược lại. Tham khảo tại https://www.ascii-code.com/. Ví dụ: Có thể chuyển đổi từ số 5 sang ký tự '5' bằng cách cộng **$48** (ký tự '0'). Ngược lại, để chuyển từ ký tự sang số thì trừ ký tự '0'.

- **Chuyển đổi số có nhiều chữ số**

Tách riêng từng số (hay ký tự số) sau đó áp dụng cách chuyển đổi trên số có 1 chữ số để chuyển qua lại giữa số và ký tự số tương ứng của nó.

Ví dụ: Chuyển chuỗi ký tự số '123' sang số nguyên: 1x100 + 2x10 + 3 = 123.

D.3 Lấy độ dài chuỗi trong section .data

Như đã trình bày ở trên, trong section .data có thể dùng để khai báo một số biến đã gán trước giá trị hoặc hằng số được dùng trong chương trình, ví dụ chuỗi output sẽ in ra màn hình. Tuy nhiên, thay vì gán cứng độ dài các chuỗi này khi dùng trong các system call, đoạn khai báo bên dưới cho phép lấy giá trị độ dài của chuỗi **rs**:

```
.section .data
rs:  .string "Max is "
rs_len = . -rs          # hằng số
```

hoặc

```
.section .data
rs:  .string "Max is "
rs_len: .int . -rs       # ô nhớ
```

E. YÊU CẦU & ĐÁNH GIÁ

Sinh viên thực hiện bài thực hành theo **nhóm (tối đa 3 SV)**, trong đó:

- Phần trên lớp: hoàn thành và báo cáo tiến độ trên lớp.
- Nộp bài thực hành trên website môn học theo quy định, gồm:
  - Các file .s chứa mã nguồn kèm chú thích chức năng của các lệnh.
  - File báo cáo tóm lược, kèm hình ảnh minh chứng kết quả thực thi.

Đặt tên thư mục nộp bài theo quy tắc:

LabX-NhomY

Tên file code .s và báo cáo đặt tên theo quy tắc:

LabX-NhomY-<yêu cầu>.s | LabX-NhomY-MSSV1-MSSV2-MSSV3.pdf

F. THAM KHẢO

[1] Linux assemblers: A comparison of GAS and NASM [Online] Available at: https://www.ibm.com/developerworks/library/l-gas-nasm/index.html

[2] Randal E. Bryant, David R. O'Hallaron (2011). *Computer System: A Programmer's Perspective (CSAPP)*

[3] Richard Blum (2005). *Professional Assembly Language*

HẾT